Laporan Praktikum Pemrograman Berorientasi Obyek (PBO) – Modul 13

Integrasi Aplikasi Berbasis Menu dan Pengujian Fungsional (Sistem Kasir Sederhana)

Nama : Rafael Rizky Luga Tanjung **Dosen** : Nirwana Haidar Hari, S.Pd., M.Kom.
NRP : 3125522007 **Kampus**: PENS PSDKU Sumenep (2026)

BAGIAN 1 – KONSEPSI AGILE, INTEGRASI MENU INTERAKTIF DAN SEPARATION OF CONCERNS

1. Latar Belakang & Analisis Agile Project Kickoff (Sprint P13): Melanjutkan pengembangan proyek Sistem Kasir Sederhana dari Pertemuan 12 dengan kerangka kerja Agile (Scrum). Fokus Sprint P13 adalah Epic: *Interactive Console Application, Separation of Concerns (SoC) & Dirty State Invariant Tracking*. Pada tahapan sebelumnya (P10–P12), pengujian masih dijalankan melalui skrip otomasi sekuensial statis tanpa interaksi pengguna langsung. Pada Sprint P13, seluruh fungsionalitas sistem (domain models, collection catalog, custom exceptions, dan file repositories) diintegrasikan ke dalam antarmuka konsol interaktif berbasis loop menu menggunakan java.util.Scanner. Prinsip SoC ditegakkan di mana class Main hanya bertugas menyajikan antarmuka dan membaca input, sementara validasi bisnis dan keunikan kode tetap didelegasikan sepenuhnya ke class domain dan pengelola.

• **Agile User Stories & Acceptance Criteria:** (1) *Sebagai Kasir*, saya ingin sistem menyediakan menu konsol interaktif yang dapat digunakan berulang kali dalam satu sesi kerja. (2) *Sebagai Kasir*, saya ingin membuka transaksi kasir baru, memilih produk, menerapkan diskon member, dan menerima pembayaran tunai. (3) *Sebagai Pemilik Toko*, saya ingin sistem memperingatkan kasir jika ingin keluar atau memuat ulang ketika ada data yang belum disimpan (Dirty Flag berubah). (4) *Sebagai Arsitek Sistem*, saya ingin validasi karakter pemisah '|' dipindahkan ke constructor objek domain (Perbaikan Wajib Modul 13).

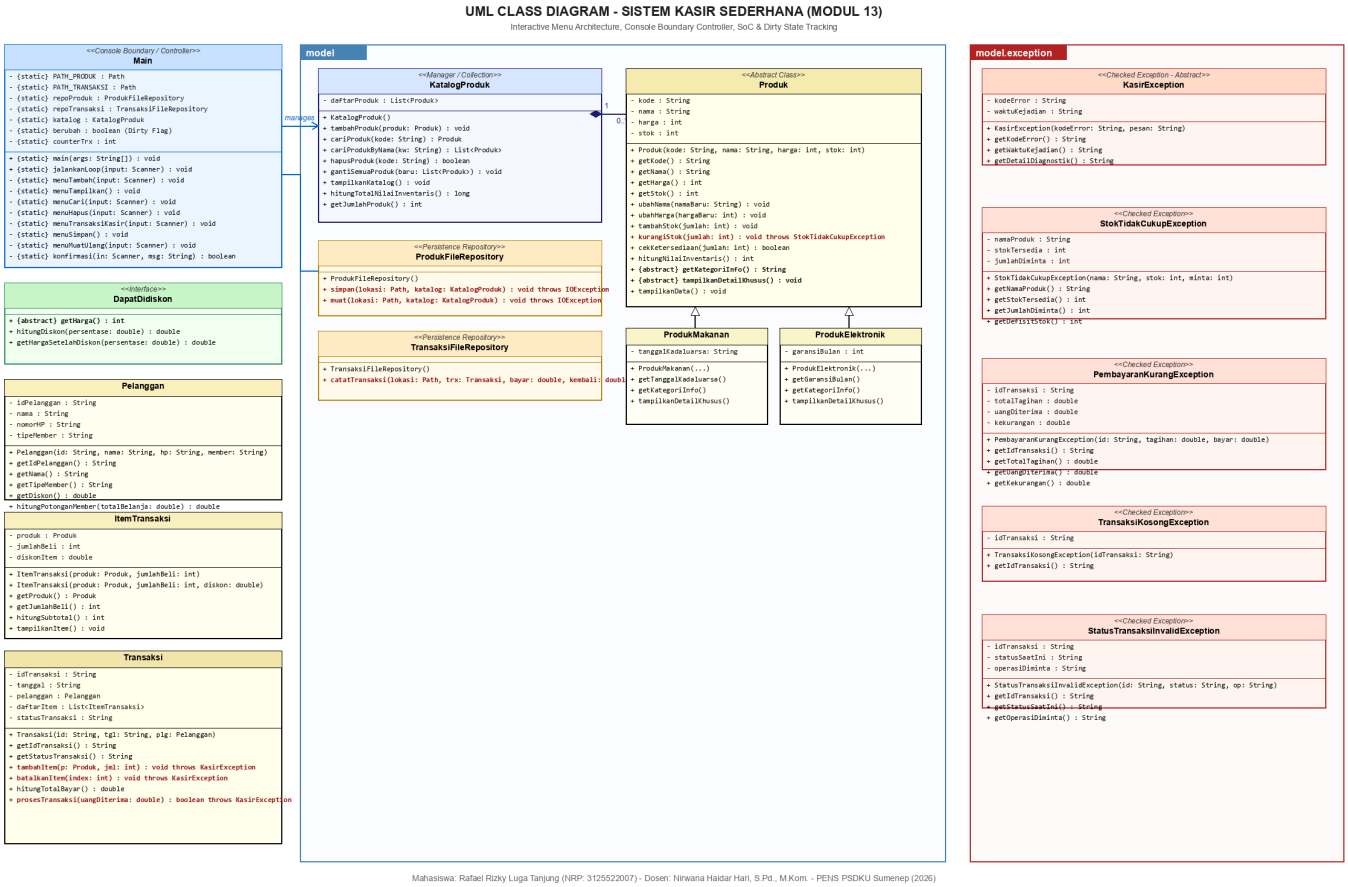
2. Desain Struktur Menu Interaktif & Pelacak Status (Dirty Flag):

Nomor Menu	Nama Layanan Menu	Method Modular Controller	Efek pada Dirty Flag (berubah)
1	Tambah Produk Baru (Makanan/Elektronik)	menuTambah(Scanner input)	Set berubah = true jika produk berhasil ditambahkan ke katalog.
2	Tampilkan Daftar Katalog Produk	menuTampilkan()	Read-only: status berubah tidak mengalami perubahan.
3	Cari Produk (by Kode / by Nama)	menuCari(Scanner input)	Read-only: status berubah tidak mengalami perubahan.
4	Hapus Produk dari Katalog	menuHapus(Scanner input)	Set berubah = true jika barang berhasil terhapus dari list.
5	Transaksi Kasir Baru & Checkout	menuTransaksiKasir(Scanner input)	Set berubah = true karena stok barang berkurang di memori RAM.
6	Simpan Data ke File (produk.txt)	menuSimpan()	Reset berubah = false karena RAM dan disk telah tersinkronisasi.
7	Muat Ulang Data dari File	menuMuatUlang(Scanner input)	Reset berubah = false setelah konfirmasi penimpaan disetujui.
0	Keluar Aplikasi	Penanganan case "0"	Wajib konfirmasi abaikan jika berubah == true.

3. Perbaikan Wajib Modul 13: Sanitasi Pemisah '|' Sejak Pembentukan Objek: Sesuai ketentuan resmi Modul 13, pemeriksaan karakter terlarang pipa (|) dan baris baru (\n, \r) yang semula diperiksa di repositori kini dipindahkan langsung ke dalam constructor Produk, ProdukMakanan, dan method ubahNama(). Dengan demikian, data yang tidak kompatibel dengan sistem persistensi berkas akan ditolak sejak pertama kali diinput di formulir menu kasir (*Fail-Fast Boundary Validation*), sehingga memori RAM toko steril dari data cacat format.

BAGIAN 2 – ARSITEKTUR KODE, UML CLASS DIAGRAM DAN TABEL PEMETAAN JAVA

4. UML Class Diagram Terbaru (Modul 13 - Interactive Menu Controller Architecture):



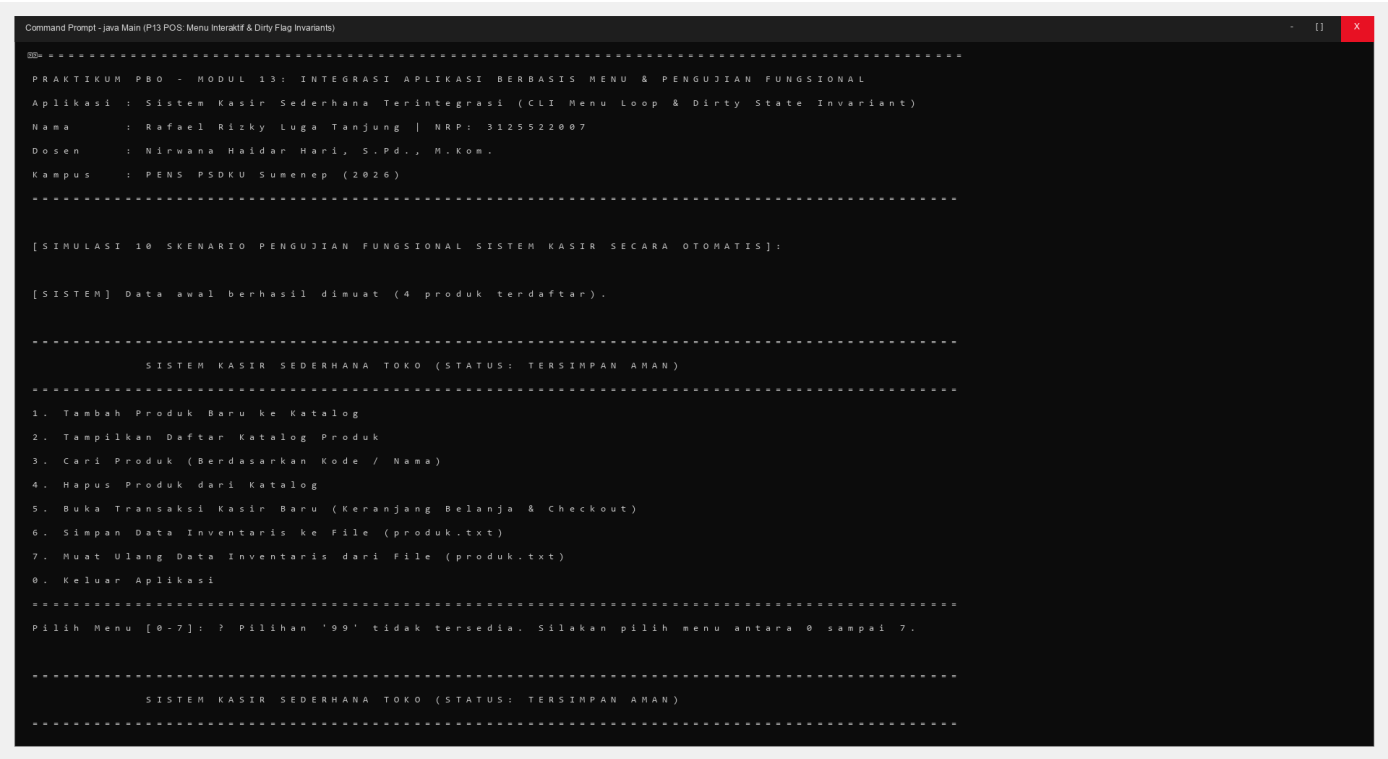
Gambar 1. UML Class Diagram Modul 13 (Boundary Controller Main dengan Dekomposisi Method Menu)

5. Tabel Pemetaan Elemen UML & Menu Controller ke Source Code Java:

Elemen UML / Controller	Notasi di Diagram UML	Implementasi pada Source Code Java
Boundary Controller	class Main <>	public class Main (mengelola siklus menu konsol dan navigasi pengguna)
Dekomposisi Method Menu	- menuTambah(input: Scanner) : void	Pemisahan logika switch-case panjang menjadi sub-method modular.
Pelacak Status Perubahan	- berubah : boolean	private static boolean berubah; (dirty flag pengaman data tak tersimpan)
Integrasi Transaksi POS	Main ..> Transaksi : creates & checkout	Main membuka Transaksi baru, menampung item, dan memproses pelunasan kasir.
Koordinasi Repository	Main ..> ProdukFileRepository : invokes	Penyimpanan dan pemuatan diarahkan melalui class repository resmi.

BAGIAN 3 – EKSEKUSI PROGRAM, SKENARIO UJI DAN EVALUASI MODUL 13

6. Bukti Eksekusi Terminal (Main.java & MainLab.java Bebas Error):



Gambar 2. Bukti Running Program Main.java Sistem Kasir Sederhana (10 Skenario Menu Interaktif Teruji 100% Sukses)



Gambar 3. Bukti Eksekusi Praktikum Dasar Modul 13 (MainLab.java: Menu Interaktif Kasus Perpustakaan)

7. Ringkasan Hasil Pengujian Fungsional (Tabel 10 Skenario Pengujian Modul 13):

No	Fokus Skenario Uji	Aksi / Input Pengujian	Hasil yang Diharapkan	Status Dirty Flag	Status
1	Pilihan Menu Salah	Input pilihan '99'	Pesan pilihan tidak tersedia, menu muncul kembali	berubah = false	**SUKSES**
2	Tampilkan Katalog	Pilih Menu 2	Tabel 4 produk awal hasil auto-load disk muncul	berubah = false	**SUKSES**
3	Tambah Makanan M03	Pilih Menu 1 -> M03 Biskuit Roma	Produk baru berhasil masuk katalog di RAM	**berubah = true**	**SUKSES**
4	Sanitasi Pemisah '|'	Input nama 'Roti | Cacat Format'	Ditolak seketika oleh constructor Produk	berubah = true (lama)	**SUKSES**
5	Cari by Kode	Pilih Menu 3 -> Kode 'E01'	Detail Mouse Wireless Ergonomis ditampilkan	berubah = true	**SUKSES**
6	Transaksi Kasir Sukses	Menu 5 -> Dr. Budi (VIP) beli M01, bayar Rp 50.000	Nota struk tercetak, kembalian sah, log transaksi dicatat	**berubah = true**	**SUKSES**

7	Uji Kurang Bayar	Menu 5 -> Tagihan besar dibayar tunai Rp 10.000	Throw PembayaranKurangException, transaksi ditolak	berubah = true	**SUKSES**
8	Proteksi Keluar Dirty	Pilih Menu 0 saat berubah=true, jawab TIDAK	Keluar dibatalkan, sistem kembali ke menu utama	berubah = true	**SUKSES**
9	Simpan ke File	Pilih Menu 6	Inventaris tersimpan ke produk.txt, flag direset	**berubah = false**	**SUKSES**
10	Keluar Sukses	Pilih Menu 0 saat berubah=false	Aplikasi tertutup bersih tanpa interupsi peringatan	Sesi berakhir	**SUKSES**

8. Jawaban Pertanyaan Evaluasi Modul 13:

1. Mengapa aturan kode unik tetap ditempatkan di class pengelola (KatalogProduk / Perpustakaan)?
Untuk menegakkan prinsip *Separation of Concerns* (SoC) dan enkapsulasi tingkat domain. Class antarmuka Main hanya bertugas membaca input teks dari konsol dan menampilkan pesan ke pengguna. Jika logika keunikan kode diletakkan di Main, maka apabila kelak sistem kasir dihubungkan ke Desktop GUI atau Web API, aturan validasi tersebut harus ditulis ulang dan rentan bocor. Dengan menaruhnya di KatalogProduk, keunikan kode dijamin terlindungi secara terpusat oleh satu otoritas pengelola koleksi.

2. Mengapa status berubah direset (menjadi false) setelah penyimpanan berhasil?
Karena operasi simpan() yang sukses menandakan bahwa seluruh data mutasi objek di memori RAM telah disinkronkan secara identik ke media fisik penyimpanan (Harddisk/SSD). Dengan mereset berubah = false, sistem mengetahui bahwa tidak ada lagi data tertinggal yang berisiko hilang, sehingga pengguna diizinkan untuk menutup aplikasi secara langsung tanpa perlu konfirmasi berulang.

3. Apa akibatnya jika status tersebut direset ketika penyimpanan gagal?
Akan terjadi fenomena *Silent Data Loss* yang sangat fatal. Jika penyimpanan ke disk gagal (misalnya karena permission ditolak, disk penuh, atau IOException) tetapi status direset menjadi false, maka sistem kasir menganggap data di disk sudah aman. Pengguna kemudian akan keluar dari aplikasi tanpa peringatan, dan seluruh transaksi atau stok baru yang ada di RAM akan musnah selamanya tanpa disadari oleh pengguna.

4. Mengapa pengujian setiap fitur perlu dilengkapi pengujian alur penggunaan secara utuh?
Pengujian unit per-fitur hanya membuktikan bahwa satu method bekerja secara terisolasi. Namun dalam dunia nyata, pengguna menjalankan rangkaian alur bersambung: memuat data awal $\rightarrow$ menambah barang $\rightarrow$ transaksi kasir $\rightarrow$ membatalkan item $\rightarrow$ menyimpan berkas $\rightarrow$ menutup aplikasi $\rightarrow$ membuka kembali di sesi berikutnya. Pengujian alur utuh (*End-to-End Functional Flow*) sangat krusial untuk memastikan bahwa transisi antar-state, sinkronisasi stok, dan persistensi berkas bekerja secara harmonis tanpa efek samping (*side-effects*).